

회원가입 집주소(address) 추가 매뉴얼

이 문서는 현재 서비스 코드에 직접 변경을 적용하지 않고, 나중에 안전하게 구현할 수 있도록 절차만 안내합니다.

1. 목표

- 회원가입 폼에서 집주소 입력을 받는다.
- DB users 테이블에 주소 컬럼을 저장한다.
- 관리자/학생 화면에서 주소를 필요 시 확인 가능하게 확장한다.
- 기존 사용자 데이터와 호환되도록 무중단에 가깝게 반영한다.

2. 변경 대상(예정)

- 백엔드
 - shared/class/app.py
- 템플릿
 - shared/class/templates/signup.html
 - (선택) shared/class/templates/admin.html
 - (선택) shared/class/templates/student.html
- DB
 - users 테이블

3. 사전 점검

1. DB 백업 수행
2. 현재 users 스키마 확인
3. 운영/개발 환경 분리 여부 확인
4. 배포 중단 가능 시간(있다면) 확정

4. DB 스키마 설계

권장 컬럼

- 컬럼명: address
- 타입: VARCHAR(255)
- NULL 허용: YES (기존 계정 호환 목적)

예시 SQL

```
ALTER TABLE users
ADD COLUMN address VARCHAR(255) NULL;
```

참고:

- 먼저 NULL 허용으로 추가한 뒤, 운영 안정화 후 NOT NULL 정책으로 전환할지 판단하는 방식이 안전합니다.

5. 백엔드 반영 절차

파일: shared/class/app.py

1. User 모델에 address 컬럼 추가
2. signup()에서 폼 데이터 address 수집
3. new_user 생성 시 address 저장

추가 위치는 `signup()` 함수의 `if request.method == 'POST':` 블록 안입니다. `new_user = User(...)` 를 만들기 직전에 주소 값을 정리해서 변수로 받은 뒤, 그 변수를 `new_user`에 넣으면 됩니다.

반영 예시 스니펫

```
raw_address = request.form.get('address', '').strip()
address = raw_address if raw_address else None

class User(db.Model):
    __tablename__ = 'users'
    student_id = db.Column(db.String(10), primary_key=True)
    login_id = db.Column(db.String(20), unique=True, nullable=False)
    password = db.Column(db.String(255), nullable=False)
    name = db.Column(db.String(20), nullable=False)
    phone = db.Column(db.String(15))
    email = db.Column(db.String(50))
    address = db.Column(db.String(255)) # 신규
    is_admin = db.Column(db.Boolean, default=False)
    grade = db.relationship('Grade', backref='user', use_list=False)
```

```
new_user = User(
    student_id=new_id,
    login_id=request.form['login_id'],
    password=hashed_pw,
    name=request.form['name'],
    phone=request.form.get('phone'),
    email=request.form.get('email'),
    address=address
)
```

6. 회원가입 화면 반영 절차

파일: shared/class/templates/signup.html

1. 기존 연락처/이메일 입력 필드 아래에 주소 입력 필드 추가
2. 필수 여부 결정
 - 선택 입력: required 미사용
 - 필수 입력: required 사용 + 서버측 검증도 추가

입력 필드 예시

```
<label for="address">집주소</label>
<input id="address" type="text" name="address" placeholder="예: 서울시 ...">
```

7. 검증/보안 가이드

1. 서버에서 길이 제한 검증
 - 예: 255자 초과 시 거부 또는 잘라내기
2. XSS 방지
 - Jinja 기본 이스케이프를 유지하고, 템플릿에서 unsafe 렌더링 금지
3. 공백 입력 처리
 - 공백만 들어온 경우 None 처리 또는 빈 문자열 정규화

예시 로직:

```
raw_address = request.form.get('address', '').strip()
address = raw_address if raw_address else None
```

8. (선택) 관리자 화면에 주소 표시

파일: shared/class/templates/admin.html

- 학생 목록 표에 주소 컬럼 추가
- 너무 길면 CSS로 줄바꿈 처리

예시 CSS 아이디어:

```
.address-cell {
    max-width: 240px;
    white-space: normal;
    word-break: break-word;
}
```

9. 테스트 체크리스트

1. 주소 없이 회원가입 성공
2. 주소 포함 회원가입 성공
3. 한글/영문/숫자/특수문자 포함 주소 저장 확인
4. 255자 경계값 테스트
5. 기존 계정 로그인/대시보드 정상 동작

6. 관리자 화면(표시 추가 시) 레이아웃 깨짐 여부

10. 롤백 플랜

1. 앱 코드 롤백
 2. DB 컬럼 유지 또는 제거 결정
- 즉시 제거 필요 시:

```
ALTER TABLE users  
DROP COLUMN address;
```

주의:

- 컬럼 제거는 데이터 손실이 발생하므로 신중히 수행

11. 권장 작업 순서(요약)

1. DB 백업
2. address 컬럼 추가(NULL 허용)
3. 모델/회원가입 폼/signup 처리 반영
4. 로컬 테스트
5. 운영 배포
6. 로그 모니터링
7. 필요 시 관리자 화면 확장

필요하면 다음 단계로, 위 절차를 기준으로 실제 적용용 패치 순서(커밋 단위)까지 세분화한 실행 가이드도 작성할 수 있습니다.